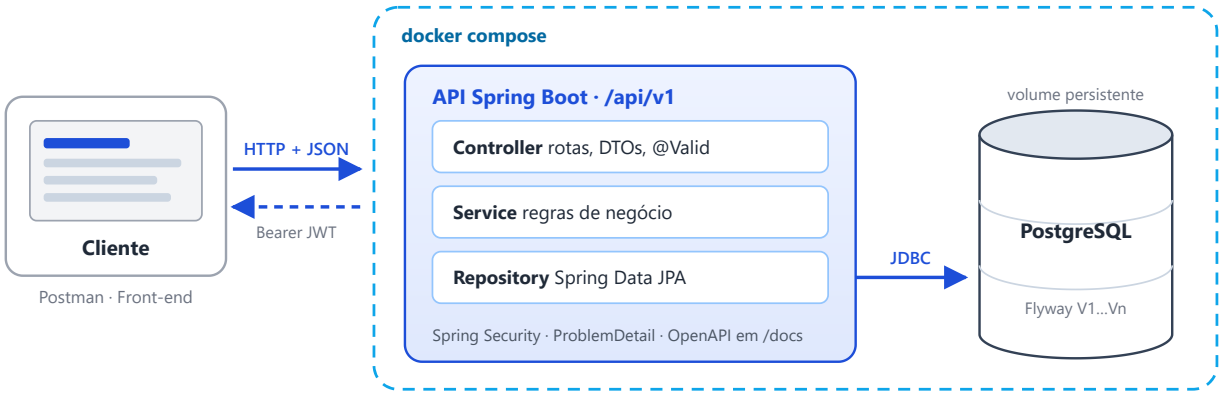
ENGENHARIA DE SOFTWARE · 6º PERÍODO · PROJETO PBL

ServiceHub API
Mapa de Construção

Especificação de banco de dados e endpoints para a refatoração do Ciclo 3

Até aqui vocês construíram a API seguindo o norte de cada aula. Este documento é o **alvo**: o schema do banco, o contrato dos endpoints e as regras de negócio que a API do ServiceHub precisa cumprir. O trabalho agora é o de qualquer engenheiro que entra num projeto existente: comparar o que existe com o que foi especificado e refatorar até chegar lá, sem jogar fora o que já funciona.



A API que vocês vão entregar: cliente, API em camadas e banco, orquestrados pelo Docker Compose.

Conteúdo

1. Como usar este mapa
2. Domínio e atores
3. Modelo de dados
4. Ciclo de vida da ordem de serviço
5. Convenções do contrato
6. Mapa de endpoints
7. Payloads de referência
8. Regras de validação
9. Regras de negócio
10. Segurança
11. Roteiro de refatoração
12. Critérios de aceite da entrega

Disciplina	Desenvolvimento de API Back-End (11296)
Instituição	UniEVANGÉLICA — Universidade Evangélica de Goiás
Professor	Prof. Esp. Eduardo Dias Pereira
Versão	1.0 — outubro de 2026

1. Como usar este mapa

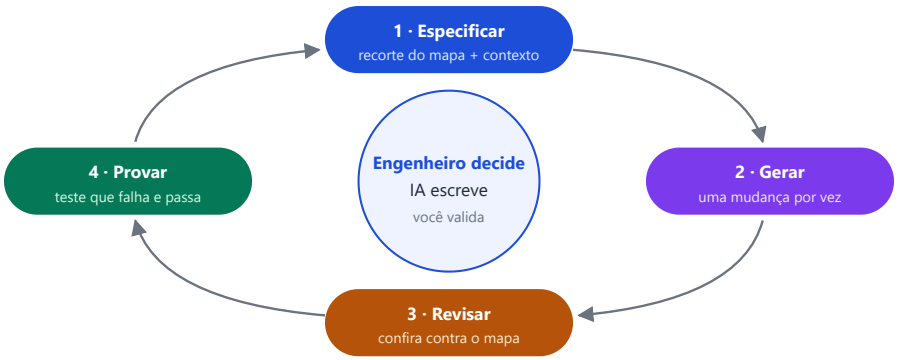
↑ voltar ao conteúdo

Em projetos reais quase ninguém começa do zero. Você recebe um código que já roda, com decisões que você não tomou, e uma especificação do que ele deveria fazer. A distância entre os dois é o seu trabalho. É isso que vamos praticar no Ciclo 3.

As quatro regras do jogo

1. **Refatore, não reescreva.** Parta da API que você já tem. Cada mudança deve deixar o projeto compilando e com os testes passando.
2. **O banco evolui por migração.** Nunca edite uma migração Flyway já aplicada. Crie `V{n+1}__descricao.sql` até o seu banco ficar igual ao schema da seção 3. Mantenha `spring.jpa.hibernate.ddl-auto=validate`.
3. **O contrato é o da seção 6.** Rotas, métodos, status codes e formato de erro são obrigatórios. Nomes internos de classes e pacotes são escolha sua.
4. **Commits pequenos e com mensagem clara.** Um commit por mudança coerente, mensagem no padrão `feat:`, `fix:`, `refactor:`, `test:`, `docs:`. O histórico do repositório faz parte da avaliação.

Trabalhando com IA neste ciclo



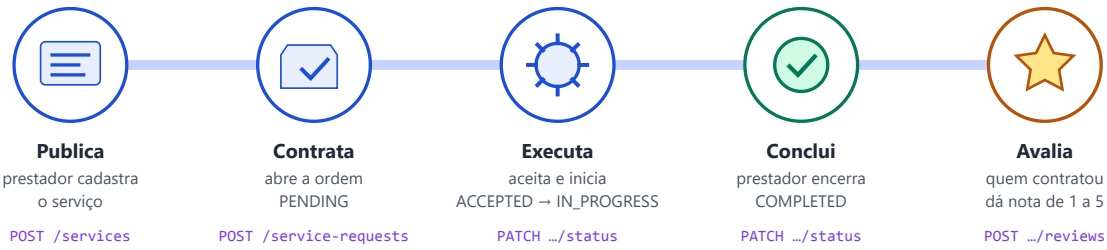
O ciclo de cada mudança: a especificação vem antes do código e o teste fecha a volta.

A IA escreve Java mais rápido que qualquer um de nós, mas não sabe o que a sua API deve fazer: esse conhecimento está neste documento. Peça **uma mudança por vez**, entregando ao assistente o trecho do mapa que interessa, e mantenha no repositório um arquivo de contexto (`CLAUDE.md`, `AGENTS.md`) com stack, convenções e decisões. Antes de aceitar o código, confira rota, status code, camada certa, DTO em vez de entidade e ausência de senha na resposta; depois, prove com um teste que falha quando a regra é quebrada.

2. Domínio e atores

↑ voltar ao conteúdo

O ServiceHub conecta pessoas que precisam de um serviço (limpeza, elétrica, suporte de TI…) a prestadores que o oferecem. O fluxo principal é: **o prestador publica um serviço → um usuário contrata (abre uma ordem) → o prestador aceita, executa e conclui → quem contratou avalia**.



A jornada principal do ServiceHub, com o endpoint que realiza cada passo.

Papel	Quem é	O que pode fazer
`CLIENT`	Pessoa que contrata serviços	Ver o catálogo, abrir ordens, cancelar as próprias ordens enquanto não iniciadas, avaliar ordens concluídas, editar o próprio perfil.
`PROVIDER`	Profissional que oferece serviços	Tudo o que o CLIENT faz (um prestador também contrata outros prestadores) e ainda: publicar, editar e desativar os próprios serviços; aceitar, iniciar, concluir ou cancelar as ordens recebidas.
`ADMIN`	Equipe da plataforma	Listar e gerenciar usuários, alterar papéis, ativar/desativar contas, intervir em qualquer serviço ou ordem. Não abre ordens.

Cada usuário tem **um único papel**, escolhido no cadastro (`CLIENT` ou `PROVIDER`). Só um ADMIN muda o papel de alguém. O primeiro ADMIN é criado pela própria aplicação no startup, a partir de variáveis de ambiente (nunca por um `INSERT` com senha dentro de uma migração versionada).

Entidade	Tabela	Significado
Usuário	`users`	Qualquer pessoa autenticável da plataforma.
Serviço	`services`	Oferta publicada por um prestador, com preço e categoria.
Ordem de serviço	`service_requests`	Contratação de um serviço por um usuário. Tem status e preço congelado.
Avaliação	`reviews`	Nota de 1 a 5 dada por quem contratou, após a conclusão.

3. Modelo de dados

↑ voltar ao conteúdo

O schema completo está no arquivo `servicehub_schema.sql`. Ele descreve o estado final; não o execute por cima do seu banco. Use-o para escrever as migrações que faltam.

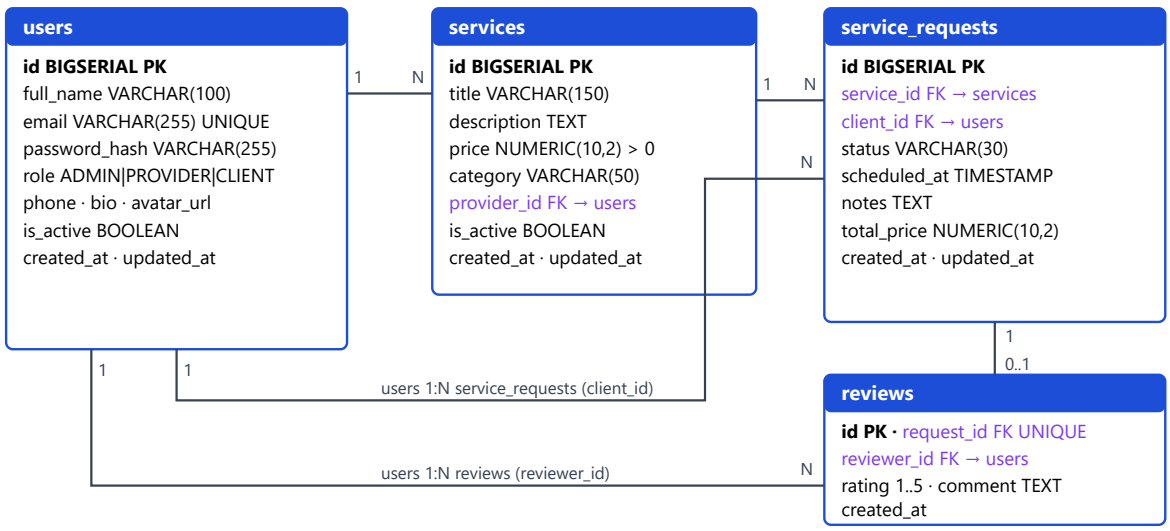


Figura 1 — Diagrama entidade-relacionamento do ServiceHub.

Decisões de modelagem que vocês precisam entender

Decisão	Por quê
Exclusão lógica (`is_active`)	Apagar um prestador apagaria o histórico de ordens e avaliações de outras pessoas. "Excluir" usuário ou serviço significa marcar `is_active = false`.
`ON DELETE RESTRICT` em todas as FKs	Coerente com a exclusão lógica: o banco recusa um `DELETE` físico que deixaria registros órfãos. Se o seu projeto tem `ON DELETE CASCADE` (vinha no material da Aula 4), troque por migração.
`total_price` na ordem	É uma cópia do preço no momento da contratação. Se o prestador reajustar o serviço amanhã, a ordem de hoje não muda.
`CHECK` em `role`, `category`, `status`, `rating`, `price`	Segunda linha de defesa. A primeira é a validação na aplicação, que devolve uma mensagem clara; o banco garante que nenhum caminho (script, bug, console) grave valor inválido.
`email = LOWER(email)`	A aplicação normaliza o e-mail (`trim` + minúsculas) antes de salvar. Sem isso, `Maria@X.com` e `maria@x.com` seriam duas contas.
`UNIQUE (request_id)` em `reviews`	No máximo uma avaliação por ordem. A aplicação verifica antes e responde 409; a constraint impede a duplicata mesmo com duas requisições simultâneas.
Enums como `VARCHAR`	Mapeie com `@Enumerated(EnumType.STRING)`, nunca `ORDINAL`: reordenar o enum Java não pode corromper os dados.

Dicionário de dados

Coluna	Tipo	Regra
users		
`full_name`	VARCHAR(100) NOT NULL	2 a 100 caracteres.
`email`	VARCHAR(255) NOT NULL UNIQUE	Formato de e-mail; minúsculas.
`password_hash`	VARCHAR(255) NOT NULL	Hash BCrypt. Nunca aparece em JSON.
`role`	VARCHAR(20) NOT NULL	`ADMIN`, `PROVIDER` ou `CLIENT`; padrão `CLIENT`.
`phone` / `bio` / `avatar_url`	VARCHAR(20) / TEXT / VARCHAR(500)	Opcionais; `bio` até 2000 caracteres na aplicação.
`is_active`	BOOLEAN NOT NULL	Padrão `true`.
services		
`title` / `description`	VARCHAR(150) / TEXT	Título 3 a 150; descrição até 2000.
`price`	NUMERIC(10,2) NOT NULL	Maior que zero. Use `BigDecimal` no Java, nunca `double`.
`category`	VARCHAR(50) NOT NULL	Um dos 21 valores do enum `ServiceCategory` (lista no SQL).
`provider_id`	BIGINT NOT NULL FK	Sempre o usuário autenticado que criou o serviço.
service_requests		
`status`	VARCHAR(30) NOT NULL	`PENDING` · `ACCEPTED` · `IN_PROGRESS` · `COMPLETED` · `CANCELLED`.
`scheduled_at` / `notes`	TIMESTAMP / TEXT	Data futura, opcional; observações até 1000.
`total_price`	NUMERIC(10,2) NOT NULL	Copiado de `services.price` na criação.
reviews		
`rating` / `comment`	SMALLINT / TEXT	Nota 1 a 5; comentário até 1000.

4. Ciclo de vida da ordem de serviço

↑ voltar ao conteúdo

Esta é a regra de negócio central do ServiceHub. O `CHECK` do banco só garante que o status é um dos cinco valores; **quem pode ir de qual estado para qual** é responsabilidade da camada de serviço (`@Service`), e cada transição precisa de teste.

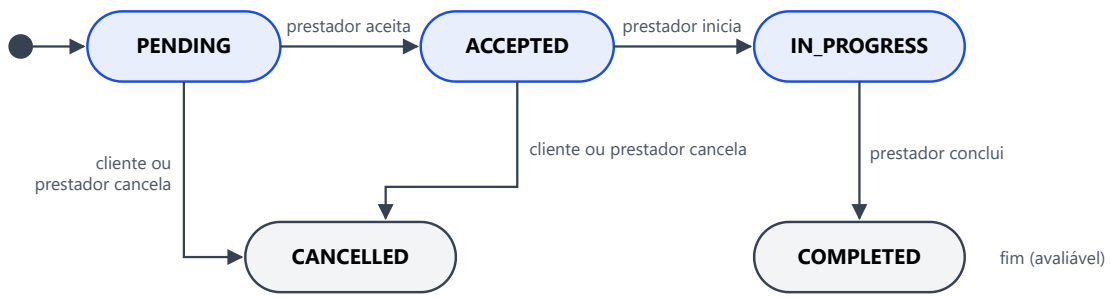


Figura 2 — Máquina de estados da ordem de serviço. COMPLETED e CANCELLED são terminais.

De	Para	Quem pode
`PENDING`	`ACCEPTED`	Prestador dono do serviço
`PENDING`	`CANCELLED`	Quem abriu a ordem, ou o prestador dono
`ACCEPTED`	`IN_PROGRESS`	Prestador dono
`ACCEPTED`	`CANCELLED`	Quem abriu a ordem, ou o prestador dono
`IN_PROGRESS`	`COMPLETED`	Prestador dono
Qualquer outra combinação		**409 Conflict**, com `detail` explicando a transição recusada (ex.: "*Não é possível mudar de COMPLETED para PENDING.*")

O ADMIN pode executar qualquer transição **válida** da tabela. Uma pessoa sem relação com a ordem recebe **403** antes mesmo de a transição ser avaliada.

5. Convenções do contrato

↑ voltar ao conteúdo

Aspecto	Regra
Prefixo e versão	Todas as rotas começam com `/api/v1` (versionamento por path, Aula 5).
Formato	JSON em UTF-8, nomes de campo em `camelCase`. Datas em ISO-8601 sem fuso: `2026-11-05T14:30:00` (horário de Brasília).
Autenticação	Header `Authorization: Bearer <token>` em toda rota que não for marcada como pública.
Criação	`201 Created`, corpo com o recurso e header `Location: /api/v1/<recurso>/{id}`.
Exclusão	`204 No Content`, sem corpo.
Listagens	Sempre paginadas: `?page=0&size=20&sort=createdAt,desc`. Padrão 20 itens, **máximo 100** (um `size=500` devolve 100, nunca erro).
Erros	Sempre `application/problem+json` no formato **ProblemDetail** (RFC 9457), com `type`, `title`, `status`, `detail`, `instance` e `timestamp`. Nunca stack trace.
Rastreio	Toda resposta tem o header `X-Request-Id`. Se o cliente enviar um válido, a API o devolve; se não, gera um UUID.
Campos desconhecidos	São ignorados. Campos que o cliente não pode definir (`id`, `role`, `providerId`, `status` na criação...) simplesmente não existem no DTO de entrada.

Página

```
{
  "content": [ { ... }, { ... } ],
  "page": {
    "size": 20,
    "number": 0,
    "totalElements": 57,
    "totalPages": 3
  }
}
```

Erro de validação (400)

```
{
  "type":
"urn:servicehub:problem:validation-error",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Um ou mais campos estão
inválidos.",
  "instance": "/api/v1/services",
  "timestamp": "2026-11-05T17:30:12Z",
  "errors": [
    { "field": "price",
      "message": "deve ser maior que 0" }
  ]
}
```

Status codes usados pela API

Código	`type`	Quando
400	`…:validation-error`	Campo inválido, JSON malformado, enum com valor desconhecido (o `detail` lista os valores aceitos).
401	`…:unauthorized`	Sem token, token inválido/expirado, credenciais erradas no login, usuário desativado.
403	`…:forbidden`	Autenticado, mas sem permissão: papel errado ou recurso de outra pessoa.
404	`…:not-found`	Recurso inexistente (ou serviço inativo, para quem não é o dono nem ADMIN).
409	`…:email-already-exists`, `…:business-rule`, `…:invalid-transition`	Requisição bem formada que viola uma regra: e-mail repetido, transição de status proibida, avaliação duplicada, exclusão com ordens ativas.
500	`…:internal-error`	Erro inesperado. `detail` genérico pedindo o `X-Request-Id`; o erro real fica só no log.

Prefixo de todos os `type`: `urn:servicehub:problem:`

6. Mapa de endpoints

↑ voltar ao conteúdo

Rotas relativas a `/api/v1`. "Dono" significa o usuário dono daquele recurso específico (verificado no service, com o usuário do token; nunca com um id vindo do corpo). Documentação interativa: Swagger UI em `/docs` e especificação OpenAPI em `/api-docs`.

6.1 Autenticação

Método	Rota	Acesso	Sucesso	Erros
POST	`/auth/register`	público	201 Usuário	400 · 409 e-mail já cadastrado
POST	`/auth/login`	público	200 Token	400 · 401 credenciais inválidas ou conta inativa
GET	`/auth/me`	autenticado	200 Usuário	401

6.2 Usuários

Método	Rota	Acesso	Sucesso	Erros
GET	`/users`	ADMIN	200 Página de Usuário	401 · 403
GET	`/users/{id}`	o próprio ou ADMIN	200 Usuário	401 · 403 · 404
PUT	`/users/{id}`	o próprio ou ADMIN	200 Usuário	400 · 401 · 403 · 404 — altera só nome, telefone, bio e avatar
DELETE	`/users/{id}`	ADMIN	204	401 · 403 · 404 · 409 ordens ativas — exclusão lógica; desativa também os serviços do usuário
PATCH	`/users/{id}/role`	ADMIN	200 Usuário	400 · 401 · 403 · 404
PATCH	`/users/{id}/active`	ADMIN	200 Usuário	400 · 401 · 403 · 404 — reativar não reativa os serviços

6.3 Serviços

Método	Rota	Acesso	Sucesso	Erros
GET	`/services`	público	200 Página de Serviço	— só ativos. Filtros: `category`, `providerId`, `minPrice`, `maxPrice`, `q` (busca no título, sem diferenciar maiúsculas)
GET	`/services/{id}`	público	200 Serviço	404 (inativo: 404 para o público, visível ao dono e ao ADMIN)
POST	`/services`	PROVIDER	201 Serviço	400 · 401 · 403 — o prestador é o usuário do token

Método	Rota	Acesso	Sucesso	Erros
PUT	`/services/{id}`	dono ou ADMIN	200 Serviço	400 · 401 · 403 · 404 — aceita `active` para reativar/desativar
DELETE	`/services/{id}`	dono ou ADMIN	204	401 · 403 · 404 · 409 ordens ativas — exclusão lógica
GET	`/services/{id}/reviews`	público	200 Página de Avaliação	404

6.4 Ordens de serviço

Método	Rota	Acesso	Sucesso	Erros
POST	`/service-requests`	CLIENT ou PROVIDER	201 Ordem (`PENDING`)	400 · 401 · 403 (ADMIN) · 404 serviço · 409 serviço inativo ou do próprio usuário
GET	`/service-requests`	autenticado	200 Página de Ordem	401 — só as ordens em que o usuário é cliente ou prestador (ADMIN vê todas). Filtros: `status` , `role=client|provider`
GET	`/service-requests/{id}`	cliente da ordem, prestador do serviço ou ADMIN	200 Ordem	401 · 403 · 404
PATCH	`/service-requests/{id}/status`	conforme seção 4	200 Ordem	400 · 401 · 403 · 404 · 409 transição inválida
POST	`/service-requests/{id}/reviews`	cliente da ordem	201 Avaliação	400 · 401 · 403 · 404 · 409 ordem não concluída ou já avaliada

6.5 Avaliações, status e operação

Método	Rota	Acesso	Sucesso	Erros
GET	`/reviews/{id}`	público	200 Avaliação	404
GET	`/info`	público	200 `{status, version, message}`	—
GET	`/actuator/health` · `/actuator/metrics` · `/actuator/prometheus` (fora de `/api/v1`)	health público; demais ADMIN	200	401 · 403 — health é usado pelo healthcheck do Docker

7. Payloads de referência

↑ voltar ao conteúdo

Exemplos completos. Os nomes de campo são parte do contrato.

POST /auth/register

```
{
  "fullName": "Maria Silva",
  "email": "maria.silva@email.com",
  "password": "Senha123",
  "role": "PROVIDER"
}
```

POST /auth/login → 200

```
// requisição
{ "email": "maria.silva@email.com",
  "password": "Senha123" }

// resposta
{ "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresIn": 86400 }
```

POST /services

```
{
  "title": "Instalação elétrica
residencial",
  "description": "Tomadas, disjuntores e
fiação",
  "price": 150.00,
  "category": "ELETRICA"
}
```

Sem `providerId`: o prestador vem do token. `PUT` aceita os mesmos campos mais `active`.

Usuário (resposta)

```
{
  "id": 7,
  "fullName": "Maria Silva",
  "email": "maria.silva@email.com",
  "role": "PROVIDER",
  "phone": "62999990000",
  "bio": "Eletricista há 12 anos",
  "avatarUrl": null,
  "active": true,
  "createdAt": "2026-11-05T14:30:00",
  "updatedAt": "2026-11-05T14:30:00"
}
```

Nunca: `password`, `passwordHash` ou qualquer campo interno do Hibernate.

Serviço (resposta)

```
{
  "id": 12,
  "title": "Instalação elétrica
residencial",
  "description": "Tomadas, disjuntores e
fiação",
  "price": 150.00,
  "category": "ELETRICA",
  "active": true,
  "providerId": 7,
  "providerName": "Maria Silva",
  "createdAt": "2026-11-05T14:35:00",
  "updatedAt": "2026-11-05T14:35:00"
}
```

POST /service-requests

```
{
  "serviceId": 12,
  "scheduledAt": "2026-11-10T09:00:00",
  "notes": "Apartamento 302, interfone 3021"
}
```

PATCH /service-requests/{id}/status

```
{ "status": "ACCEPTED" }
```

POST /service-requests/{id}/reviews

```
{ "rating": 5,
  "comment": "Pontual e caprichosa." }
```

Ordem (resposta)

```
{
  "id": 31,
  "serviceId": 12,
  "serviceTitle": "Instalação elétrica
residencial",
  "clientId": 9,
  "clientName": "João Pereira",
  "providerId": 7,
  "status": "PENDING",
  "totalPrice": 150.00,
  "scheduledAt": "2026-11-10T09:00:00",
  "notes": "Apartamento 302, interfone
3021",
  "createdAt": "2026-11-05T15:00:00",
  "updatedAt": "2026-11-05T15:00:00"
}
```

Avaliação (resposta)

```
{ "id": 4, "requestId": 31, "serviceId": 12,
  "reviewerId": 9, "reviewerName": "João
Pereira",
  "rating": 5, "comment": "Pontual e
caprichosa.",
  "createdAt": "2026-11-10T12:10:00" }
```

Exemplo de regra violada (409)

```
{
  "type": "urn:servicehub:problem:invalid-transition",
  "title": "Transição de status inválida",
  "status": 409,
  "detail": "Não é possível mudar de COMPLETED para PENDING.",
  "instance": "/api/v1/service-requests/31/status",
  "timestamp": "2026-11-10T12:15:40Z"
}
```

8. Regras de validação

↑ voltar ao conteúdo

Validação declarativa com Bean Validation (`@Valid` no controller, anotações no DTO). Mensagens em português. Violações respondem 400 com a lista `errors`.

DTO	Campo	Regra
Cadastro de usuário	`fullName`	obrigatório, 2 a 100 caracteres
	`email`	obrigatório, e-mail válido, até 255; normalizado (`trim` + minúsculas)
	`password`	obrigatório, 8 a 72 caracteres, ao menos uma letra maiúscula e um número (`^(?=.*[A-Z])(?=.*\d).+$`)
	`role`	obrigatório, `CLIENT` ou `PROVIDER` (`ADMIN` → 400)
Atualização de usuário	`fullName`	obrigatório, 2 a 100
	`phone` · `bio` · `avatarUrl`	opcionais; até 20 · 2000 · 500
Serviço	`title`	obrigatório, 3 a 150
	`description`	opcional, até 2000
	`price`	obrigatório, de 0,01 a 99.999.999,99
	`category`	obrigatório, valor do enum
Ordem de serviço	`serviceId`	obrigatório, positivo
	`scheduledAt`	opcional, no futuro
	`notes`	opcional, até 1000
Mudança de status	`status`	obrigatório, valor do enum
Avaliação	`rating`	obrigatório, inteiro de 1 a 5
	`comment`	opcional, até 1000
Papel / ativação	`role` · `active`	obrigatórios; `CLIENT|PROVIDER|ADMIN` · `true|false`

> **Sanitização (Aula 8):** campos de texto livre (`description`, `notes`, `comment`, `bio`) passam por escape de HTML na camada de serviço antes de gravar. Toda consulta usa parâmetros (JPA, `@Query` com `:param`); nunca concatenação de strings em SQL.

9. Regras de negócio

↑ voltar ao conteúdo

Cada regra abaixo deve ter **pelo menos um teste** que falha quando ela é quebrada. Citem o código da regra no nome ou no `@DisplayName` do teste.

Código	Regra	Resposta
RN01	E-mail é único na plataforma, comparado sem diferenciar maiúsculas.	409
RN02	Senha é gravada só como hash BCrypt e nunca sai em resposta nenhuma.	—
RN03	No cadastro, o papel só pode ser `CLIENT` ou `PROVIDER`. Só um ADMIN altera papéis.	400 / 403
RN04	O prestador de um serviço é sempre o usuário autenticado que o criou.	—
RN05	Só o dono ou um ADMIN altera ou desativa um serviço.	403
RN06	CLIENT e PROVIDER podem abrir ordens; ninguém contrata o próprio serviço; o serviço precisa estar ativo.	409
RN07	`totalPrice` da ordem é o preço do serviço no momento da criação e não muda depois.	—
RN08	Mudanças de status seguem a tabela da seção 4.	403 / 409
RN09	Só quem abriu a ordem avalia, só depois de `COMPLETED`, uma única vez.	403 / 409
RN10	Usuário ou serviço com ordens `PENDING`, `ACCEPTED` ou `IN_PROGRESS` não pode ser excluído.	409
RN11	Desativar um usuário desativa os serviços dele. Usuário inativo não faz login e o token que ele já tinha deixa de valer.	401
RN12	Serviço inativo não aparece nas listagens e responde 404 ao público; dono e ADMIN continuam vendo.	404
RN13	Uma pessoa só vê o próprio perfil e as ordens em que participa (ADMIN vê tudo).	403

10. Segurança

↑ voltar ao conteúdo

Autenticação (Aulas 8 e 9)

- Spring Security com `SecurityFilterChain`, sessão `STATELESS`, CSRF desabilitado.
- JWT assinado (HS256), expiração de 24 h. Claims: `sub` (e-mail), `uid` (id), `roles`, `iat`, `exp`.
- Segredo do JWT vem de variável de ambiente, com pelo menos 32 bytes. Sem segredo, a aplicação não sobe.
- A cada requisição, o usuário do token é conferido no banco: se estiver inativo, 401 (RN11).

Autorização

- `@EnableMethodSecurity` e `@PreAuthorize` para regras por papel.
- Regras de dono (BOLA, OWASP API1) verificadas no service com o usuário autenticado.
- Mass assignment (OWASP API3): DTOs de entrada não têm `id`, `role`, `providerId`, `clientId`.
- Excessive data exposure: só DTOs de resposta; nunca entidades JPA.

Configuração

- CORS com origens explícitas vindas de variável de ambiente; nunca `*` com credenciais.
- `server.error.include-stacktrace=never`.
- Swagger UI com botão *Authorize* (esquema bearer JWT).
- Nenhum segredo no Git: `.env` no `.gitignore`, `.env.example` versionado.

Rotas públicas

`POST /auth/register`, `POST /auth/login`, `GET /services`, `GET /services/{id}`, `GET /services/{id}/reviews`, `GET /reviews/{id}`, `GET /info`, `/actuator/health`, `/docs`, `/api-docs`.

Todo o resto exige token. Escrevam um teste que percorre as rotas protegidas e confirma 401 sem token.

11. Roteiro de refatoração

↑ voltar ao conteúdo

Ordem sugerida. Cada etapa termina com testes passando, commit e, de preferência, uma tag (`git tag etapa-1` ...). Assim vocês conseguem mostrar na banca a evolução com `git diff`.

Etapa	Foco	O que fazer	Aulas
1	Diagnóstico	Compare seu projeto com este mapa e escreva um `docs/diagnostico.md`: o que já atende, o que falta, o que contraria. É a sua lista de tarefas.	—
2	Banco	Migrações novas até o schema da seção 3 (papel, `is_active`, `updated_at`, `RESTRICT`, `CHECK`s). Entidades JPA validando com `ddl-auto=validate`.	4
3	Contrato	`/api/v1`, DTOs de entrada e saída, `ResponseEntity` com `Location`, paginação, `@RestControllerAdvice` com ProblemDetail, Bean Validation.	2, 5, 6, 8
4	Segurança	BCrypt, cadastro e login com JWT, `/auth/me`, papéis, regras de dono, CORS, admin inicial por variável de ambiente.	8, 9
5	Domínio	Ordens de serviço com a máquina de estados, avaliações, exclusão lógica com as regras RN10–RN12.	4, 9
6	Testes	Unitários das regras (Mockito), fatia web (`@WebMvcTest`), integração com PostgreSQL real (Testcontainers). Um teste por regra RN.	10
7	Documentação	OpenAPI com todos os status codes da seção 6, exemplos reais e erros documentados com o schema ProblemDetail. README atualizado.	5
8	Ambiente	Dockerfile multi-stage, Compose com healthcheck, `depends_on: service_healthy`, volume e `.env`; pipeline no GitHub Actions rodando `mvn verify`.	11, 12, 14

> **Armadilha comum:** mudar a entidade JPA e esquecer a migração. Com `ddl-auto=validate` a aplicação não sobe e o erro diz exatamente qual coluna falta. Leiam a mensagem antes de pedir ajuda (à IA ou ao professor).

12. Critérios de aceite da entrega

↑ voltar ao conteúdo

A banca vai conferir item a item. Usem como checklist antes de entregar.

Funcionamento

☐ `docker compose up -d --build` sobe API e PostgreSQL sem passo manual.

☐ Dados persistem após `docker compose down` e `up`.

☐ Fluxo completo funciona: cadastro de prestador e cliente → login → serviço → ordem → aceitar → iniciar → concluir → avaliar.

☐ Todas as rotas da seção 6 com os status codes indicados.

☐ Erros sempre em ProblemDetail; nenhum 500 para erro do cliente.

Banco

☐ Schema igual ao da seção 3, alcançado por migrações Flyway.

☐ Nenhuma migração antiga editada.

Qualidade

☐ Ao menos um teste por regra RN01–RN13.

☐ Teste de integração com PostgreSQL real.

☐ Teste que confirma 401 nas rotas protegidas sem token.

☐ Pipeline do GitHub Actions verde no último commit.

☐ Swagger em `/docs` coerente com o que a API faz.

Processo

☐ `docs/diagnostico.md` da etapa 1.

☐ Histórico com commits pequenos e mensagens claras.

☐ README com como rodar, como testar e variáveis de ambiente.

☐ `.env.example` versionado; nenhum segredo no repositório.

☐ Ao menos três decisões técnicas registradas (problema → decisão → consequência).

> **Você é avaliado pelas decisões, não pela digitação.** Na banca, espere perguntas como "por que este endpoint responde 409 e não 400?" ou "onde a sua API impede que um cliente veja a ordem de outro?". A resposta está neste mapa e no seu código.

> **Na banca:** escolham uma regra de negócio, mostrem o teste que a protege, quebrem a regra no código ao vivo e mostrem o teste falhando. Isso prova que vocês entendem o que a IA escreveu.